

7장(BERT~) 개념정리 (제공한 내용 기반)

1) BERT

1.1 BERT 개요

- **BERT(Bidirectional Encoder Representations from Transformers)**: 2018년 구글이 발표한 트랜스포머 기반 언어모델.
- **GPT와의 큰 차이**
 - GPT: 트랜스포머 **디코더 기반**(한 방향(주로 왼→오) 예측 중심)
 - BERT: 트랜스포머 **인코더 기반** + ****양방향 문맥(좌/우 모두)****을 동시에 활용
- **양방향 문맥을 본다는 의미**: 어떤 단어를 이해/예측할 때 앞 단어뿐 아니라 뒤 단어 정보도 함께 참고하므로 **문맥 파악이 더 정확해지고**, 자연어 이해(NLU) 성격의 작업에서 강함.
- **사전학습(pretraining)이 완료된 모델**을 활용하기 때문에, 보통은 **전이학습(미세조정, fine-tuning)** 형태로 사용 → 데이터가 많지 않아도 성능을 잘 낼 수 있다는 장점.

1.2 BERT 입력 구성과 특수 토큰

BERT는 모델에 넣기 전에 토큰화(tokenization) 과정이 필요하고, 특히 다음 특수 토큰이 핵심입니다.

- **[CLS]**
 - 문장 맨 앞에 붙는 토큰
 - 문장 전체를 대표하는 “요약 표현” 역할로 활용됨
 - 감정 분류(긍/부정), 문장 분류 같은 **분류 태스크에서 핵심**(일종의 대표 벡터로 쓰임)
- **[SEP]**
 - 문장 경계를 나타내는 토큰
 - 입력이 두 문장(A,B)로 구성될 때 문장 사이 구분에 사용
- **[MASK]**
 - 사전학습에서 일부 단어를 가리고 맞추게 만들 때 사용하는 토큰

- BERT의 핵심 사전학습 목표(MLM)에서 사용

1.3 [MASK] 적용 규칙(사전학습 시)

- 전체 토큰 중 **15%**를 대상으로 "맞춰야 할 대상"으로 선택
- 그 15% 중
 - **80%**는 실제로 **[MASK]**로 바꿈
 - **10%**는 랜덤 단어로 바꿈
 - **10%**는 그대로 유지
- 이렇게 섞는 이유(핵심 포인트): 항상 [MASK]만 보면 학습/추론 시 괴리가 생기기 쉬워서, 모델이 "진짜 토큰이 들어오는 상황"도 같이 견디게 만드는 장치라고 이해하면 됨.

1.4 임베딩 구성(입력 벡터가 어떻게 만들어지는가)

BERT는 입력 토큰을 그냥 임베딩 하나로만 넣는 게 아니라, 세 가지 임베딩을 더해서 최종 입력 표현을 만듭니다.

- 토큰 임베딩: 단어(토큰) 자체의 의미 벡터
- 위치 임베딩: 토큰이 문장 내 몇 번째 위치인지 반영
- 문장 구분 임베딩(세그먼트 임베딩): 두 문장 입력(A/B)일 때 "이 토큰이 A 문장인지 B 문장인지" 구분

1.5 BERT 사전학습 목표: MLM과 NSP

BERT는 사전학습을 다음 두 과제로 진행합니다.

- **MLM (Masked Language Modeling)**
 - 문장 일부 단어를 가리고([MASK] 등), 그 단어를 맞추게 함
 - 양방향 문맥을 활용하여 "앞/뒤 문맥 기반 추론" 능력을 키움
 - 결과적으로 문맥 이해 능력이 강화됨
- **NSP (Next Sentence Prediction)**
 - 두 문장(A,B)을 주고, B가 A 다음 문장인지(관련 있음) 아니면 무관한지(관련 없음) 맞추는 과제
 - 문장 간 관계/연결성 학습에 도움 → 문장쌍 기반 태스크에 유리하다는 흐

2) BERT 실습 흐름(네이버 영화 리뷰 감정 분석 예시)

2.1 데이터 로드 및 분할

- Korpora의 **nsmc**(네이버 영화 리뷰 감정 분석) 데이터 사용
- 샘플링 후 train/valid/test로 분할해서 학습-검증-테스트 진행

2.2 토큰나이징/전처리 방식

- BertTokenizer로 텍스트를 토큰화하고,
 - padding="longest"로 배치 내 가장 긴 문장에 맞춰 패딩
 - truncation=True로 너무 긴 문장 자르기
 - return_tensors="pt"로 바로 텐서 형태 반환
- 결과로 input_ids, attention_mask를 만들고 라벨을 텐서로 구성해 TensorDataset으로 묶음

2.3 collate_fn을 안 쓰는 이유(작성 내용 기반 정리)

- collate_fn을 쓰면 배치마다 동적 전처리가 가능하지만 속도/구현 복잡도 측면 trade-off가 있음
- 여기서는 미리 토큰나이징 후 TensorDataset을 만들어 사용하는 방식 선택

2.4 모델/학습

- BertForSequenceClassification로 **문장 분류 모델** 구성(num_labels=2)
- 옵티마이저는 AdamW
- 학습은
 - forward에 input_ids, attention_mask, labels를 넣어 loss를 얻고
 - backward → optimizer.step으로 업데이트
- 검증은 no_grad + eval 모드에서 loss와 accuracy 계산

3) BART

3.1 BART 개요

- **BART(Bidirectional Auto-Regressive Transformer)**: 2019년 메타 FAIR에서 발표
- 구조는 **인코더-디코더 결합(Seq2Seq)**
→ BERT처럼 인코더만 쓰는 것도, GPT처럼 디코더만 쓰는 것도 아니라 둘을 함께 사용
- 사전학습은 **노이즈 제거 오토인코더(Denoising Autoencoder)** 방식
 - 입력 문장에 일부러 "노이즈(변형)"를 넣고
 - 원래 문장을 복원하도록 학습
- 노이즈 예시: 토큰 마스킹/삭제, 문장 순열, 회전, 텍스트 채우기 등
- 인코더가 입력을 이해한 뒤, 그 정보를 디코더에 전달해서 **더 정교한 생성**이 가능해짐
→ 요약, 번역, 생성형 태스크에 잘 맞는 구조

3.2 BART 실습 흐름(뉴스 요약)

- 데이터: argilla/news-summary
 - 입력(text)과 정답 요약(prediction)을 준비
 - 전처리:
 - 입력은 토큰나이징 후 input_ids, attention_mask
 - 정답 요약은 토큰 id로 만들고 길이를 맞추기 위해 pad_sequence 사용
 - 패딩을 -100으로 두는 이유: loss 계산에서 무시되도록 하기 위함(라벨 마스킹)
 - 모델:
 - BartForConditionalGeneration
 - 평가:
 - 생성된 요약과 정답 요약의 유사도를 **ROUGE**로 측정
 - ROUGE는 생성/요약 태스크에서 흔히 쓰는 평가 지표(토큰 N-gram 기반 유사도)
-

4) ELECTRA

4.1 개요 및 BERT와 차이

- **ELECTRA(Efficiently Learning an Encoder that Classifies Token Replacements Accurately)**
 - BERT처럼 [MASK]로 가려놓고 맞추는 방식(MLM)을 그대로 쓰지 않고,
 - **Generator(생성자) + Discriminator(판별자)** 구조를 사용
 - 직관적으로 GAN과 유사한 느낌:
 - 생성자는 일부 토큰을 "그럴듯한 토큰"으로 바꾸어 끼워 넣고
 - 판별자는 각 토큰이 "원본인지/바뀐 것인지"를 맞추는 방식
 - 장점(작성 내용 기반): BERT 대비 더 **효율적 학습**, 모델이 더 **가볍고 빠르게** 운용 가능
-

5) T5

5.1 T5 개요

- **T5(Text-to-Text Transfer Transformer)**: 2019년 구글 발표
- 핵심 컨셉: **입력도 텍스트, 출력도 텍스트로 통일(Text-to-Text)**
 - 분류도 "label: positive" 같은 텍스트로, 요약도 텍스트로, 번역도 텍스트로 ...
 - 다양한 태스크를 하나의 형태로 다루기 쉬움(유연/확장성)
- 사전학습:
 - 입력 문장 일부를 마스킹하고 복원하는 방식
 - **Sentinel Token**(<extra_id_0>, <extra_id_1> ...)을 사용해 "마스크 구간"을 표시

5.2 T5 실습 흐름(뉴스 요약)

- 데이터는 BART와 동일한 뉴스 요약 데이터 사용
- 입력에 "summarize: " 프롬프트를 붙이는 구조
- 전처리:

- source/target 모두 max_length로 길이 통일(padding/truncation)
- 학습 시 중요한 처리:
 - decoder 입력은 target에서 마지막 토큰을 제외(예측 대상이므로)
 - labels는 target에서 첫 토큰을 제외(이미 입력으로 준 부분)
 - padding 토큰은 loss에서 제외되도록 -100 처리
- 생성(generate)에서 자주 쓰는 옵션(작성 내용 기반)
 - num_beams: beam search에서 후보를 몇 개 유지할지(빔 크기)
 - repetition_penalty: 반복 억제
 - length_penalty: 길이 선호 조정
 - early_stopping: eos가 나오면 조기 종료